

LunarCore Agent

细致代码文档

v2.3.0 源码架构与实现详解

2026-08-11

LunarCore Agent Project

文档版本	v2.3.0
文档日期	2026-08-11
适用项目	investment-ai (LunarCore Agent)
技术栈	React 19 + Electron 33 + Python FastAPI
文档类型	源码架构与实现详解

第一章 项目目录结构

LunarCore Agent (investment-ai) 是一个集成了 AI 对话、量化回测、实时行情、数据采集、知识库管理、计算机控制等功能于一体的跨平台桌面应用。项目采用 React 19 + Electron 33 + Python FastAPI 的三层架构，前端使用 TypeScript 和 Tailwind CSS 构建，后端量化服务使用 Python vectorbt 和 LightGBM。

整个项目目录分为四大模块：src/（前端源码）、electron/（Electron 主进程）、python/（Python 后端服务）、以及根目录的构建配置文件。下表展示了完整的目录结构。

路径	说明
investment-ai/	项目根目录
src/	前端源码目录
pages/	29 个页面组件 (Chat/Quant/Market 等)
components/	4 个 UI 组件 (Sidebar/Chart/Background/ModelSelector)
lib/	75 个核心库文件 (状态管理/路由/数据源/回测等)
main.tsx	React 应用入口
App.tsx	应用根组件 (状态驱动路由)
index.css	全局样式 (Tailwind 指令)
vite-env.d.ts	Vite 类型声明
electron/	Electron 主进程目录
main.cjs	主进程入口 (窗口创建/生命周期管理)
preload.cjs	预加载脚本 (IPC 桥接层)
cua-bridge.cjs	计算机控制桥 (鼠标/键盘/截屏)
ipc-ollama.cjs	Ollama 本地模型 IPC
ipc-market.cjs	行情数据 IPC (东方财富/腾讯/新浪)
ipc-kb.cjs	知识库 IPC (LLM Wiki/AnythingLLM)
ipc-vbt.cjs	VBT 服务 IPC
scheduler.cjs	交易调度器 (node-schedule)
vbt-manager.cjs	VBT 进程管理器 (环境检测/自动安装)

路径	说明
python/	Python 后端服务目录
vbt_server.py	FastAPI 量化服务（端口 17897）
lgbm_factors.py	LightGBM 因子模型
Kronos/	Kronos AI 预测模型（Tokenizer + Predictor）
requirements.txt	Python 依赖清单
setup.sh / setup.bat	环境安装脚本
package.json	Node.js 依赖与脚本配置
tailwind.config.js	Tailwind CSS 配置
vite.config.ts	Vite 构建配置（端口 5200）
tsconfig.json	TypeScript 编译配置

技术栈概览

层级	技术	版本	用途
前端框架	React	19.2.7	UI 渲染与状态管理
状态管理	Zustand	5.0.14	全局状态（50+ 字段持久化）
构建工具	Vite	8.1.1	开发服务器与打包
样式系统	Tailwind CSS	3.4.19	原子化 CSS + 自定义主题
图表库	ECharts	6.1.0	K 线/回测/因子可视化
桌面框架	Electron	33.4.11	跨平台桌面应用容器
后端框架	FastAPI	latest	Python 量化服务 API
量化引擎	vectorbt	latest	向量化策略回测
ML 框架	LightGBM	latest	因子模型训练与预测
AI 预测	Kronos	AAAI 2026	K 线基础模型零样本预测
类型系统	TypeScript	6.0.2	静态类型检查
测试框架	Vitest	4.1.10	单元测试与覆盖率

架构特点

- 三层解耦：前端（React）通过 IPC 桥与 Electron 主进程通信，主进程再通过 HTTP 调用 Python 后端服务，各层职责清晰。
- 双通道数据：实时行情支持 Electron IPC（无跨域）和浏览器直连两种模式，开发环境灵活切换。
- 自动降级：数据源管理器支持 30+ 数据源注册，按优先级自动切换，连续失败 5 次触发熔断。
- 进程托管：VBT Python 服务由 Electron 主进程自动管理，包含环境检测、自动安装、健康监控、优雅退出全流程。
- AI 增强：集成 9 阶段模型路由引擎、5 阶段自学习技能系统、Agent 工具调用循环，实现智能化的金融分析。

第二章 核心库模块详解（src/lib/）

src/lib/ 目录包含 75 个核心库文件，是整个应用的业务逻辑核心。这些模块涵盖了状态管理、AI 模型路由、计算机控制、量化回测、数据源管理、实时行情、自学习技能、AI 预测、知识库等全部功能域。本章将逐一深入解析每个核心模块的设计理念、类型定义、关键函数和代码实现。

2.1 store.ts — 全局状态管理

技术栈：Zustand 5.0 + persist 中间件 | 状态字段：50+ | 持久化：localStorage

模块用途

store.ts 是整个应用的状态中枢，使用 Zustand 创建全局 store 并通过 persist 中间件实现 localStorage 持久化。该模块管理着应用的所有核心状态，包括模型配置、路由规则、会话消息、自选股列表、持仓数据、量化设置、知识库端点、插件配置等 50 余个状态字段。

核心类型定义

以下是 store.ts 中定义的核心类型与接口：

```
// 枚举
type QuantMode = 'single' | 'ai' | 'multi' | 'vbt' | 'portfolio' | 'framework'

// 常量
const APP_VERSION = 'v2.3.0'
const APP_ID = 'lunarcore.agent'
const APP_NAME = 'LunarCore Agent'

// 枚举
type ModelType = 'api' | 'local' | 'aggregated'
type ModelProvider = 'openai' | 'anthropic' | 'deepseek' | 'minimax' | 'qwen' |
'zhipu'
| 'siliconflow' | 'kimi' | 'gemma' | 'ollama' | 'lmstudio' | 'custom' | 'aggregated'
| 'moonshot' | 'yi' | 'baichuan' | 'stepfun' | 'google'

interface ModelConfig {
  id: string
  name: string
  type: ModelType
  provider: ModelProvider
  endpoint: string
  apiKey?: string
  modelName: string
  enabled: boolean
  capabilities: string[] // ['chat','vision','code','embedding','analysis','summary']
  costPer1K?: number // 1K token 费用
  contextWindow?: number // 上下文窗口
  children?: string[] // 子模型 ID
  priority: number // 优先级
  maxTokens?: number
  temperature?: number
}

// 枚举
interface CascadeConfig {
  enabled: boolean
  maxRetries: number
  memoryEnabled: boolean // 内存缓存
  degradationStrategy: 'fallback' | 'round-robin' | 'cost-first' | 'quality-first'
}

// 枚举
interface VersionLog {
  version: string
  date: string
  title: string
}
```

```
changes: string[]
}

// 插件配置
type PluginType = 'skill' | 'mcp'
type PermissionLevel = 'full' | 'read-only' | 'execute-only' | 'disabled'
interface PluginPermission {
  network: boolean
  fileSystem: boolean
}
```

Zustand Store 创建与 persist 配置

store 使用 Zustand 的 create 函数创建，并通过 persist 中间件实现状态持久化。持久化配置中包含自定义的 merge 函数，用于处理版本迁移逻辑：

```

import { create } from 'zustand'
import { persist } from 'zustand/middleware'

export const useStore = create<AppState>()(
  persist(
    (set, get) => ({
      // ===== 初始化 =====
      page: 'overview',
      sidebarCollapsed: false,
      watchList: DEFAULT_WATCH,
      holdings: [],
      messages: [],
      activities: [],

      // ===== 模型 =====
      models: [],
      activeModelId: null,
      routingRules: [],
      cascadeConfig: { enabled: true, maxRetries: 3, memoryEnabled: true,
        degradationStrategy: 'fallback' },

      // ===== 模式 =====
      quantMode: 'single',
      vbtHealth: null,
      kronosConfig: DEFAULT_KRONOS_CONFIG,

      // ===== Actions =====
      setPage: (page) => set({ page }),
      addHolding: (h) => set((s) => ({ holdings: [...s.holdings, h] })),
      updateModel: (id, patch) => set((s) => ({
        models: s.models.map((m) => m.id === id ? { ...m, ...patch } : m)
      })),
      // ... 50+ 模型 actions
    }),
    {
      name: 'agentcore-store',
      version: 23, // 版本号
      // 持久化配置
      merge: (persisted, current) => {
        const p = persisted as Partial<AppState>
        // 持久化 endpoint 模型
        if (p.models) {
          p.models = p.models.map(m => migrateEndpoint(m))
        }
        // 版本日志
        if (!p.versionLogs || p.versionLogs.length === 0) {
          p.versionLogs = CURRENT_VERSION_LOGS
        }
        return { ...current, ...p }
      },
    }
  )
)

```

持久化合并逻辑详解

`persist` 中间件的 `merge` 函数是版本兼容的关键。当用户从旧版本升级时，`merge` 函数负责将旧的持久化状态与新版本的默认状态进行智能合并：

- 模型端点迁移：自动将旧版 API 端点更新为新版地址，避免用户手动重新配置。
- 版本日志强制覆盖：每次发布新版本时，版本更新日志会被强制覆盖到最新，确保用户看到最新变更。
- 新增字段默认值：新版本添加的状态字段如果在持久化数据中不存在，自动使用默认值填充。
- 数组去重：自选股列表、持仓列表等数组类型在合并时进行去重处理，防止数据重复。

2.2 modelRouter.ts — AI 模型路由引擎

9 阶段路由管线 | 会话级记忆缓存 | 4 种降级策略 | 路由日志系统

模块用途

`modelRouter.ts` 是 AI 模型调用的智能路由引擎。当用户发送消息时，路由引擎会经过 9 个阶段的处理管线，从多个已配置的模型中选择最合适的模型进行响应。该模块还集成了 `SkillClaw` 自学习技能注入和 `Cua` 计算机控制工具提示，实现端到端的 AI Agent 能力。

9 阶段路由管线

阶段	名称	说明
1	detect	任务类型检测：分析用户输入，判断是 chat/analysis/code/vision/summary
2	filter	模型筛选：根据任务类型和模型能力 (capabilities) 过滤可用模型
3	memory	会话记忆：查询 sessionMemory 三级索引，复用已验证的能力档位
4	preference	偏好计算：根据用户配置的权重 (weight) 计算模型偏好分数
5	rule	规则匹配：检查用户定义的路由规则 (RoutingRule)，强制指定模型
6	strategy	策略选择：根据 degradationStrategy 选择降级策略
7	cascade	级联探测：如首选模型失败，按级联配置尝试备选模型
8	model	模型调用：调用选定的模型 API (流式响应)
9	result	结果返回：记录路由日志，更新会话记忆，返回响应

会话级记忆缓存

会话级记忆缓存是路由引擎的核心优化。通过三级索引结构（sessionId -> taskType -> modelId），引擎可以记住同一会话内已验证的模型能力，避免在 Agent 多步调用时重复探测：

```
// 三级索引结构 sessionId -> taskType -> modelId -> capability
const sessionMemory = new Map<string, Map<string, Map<string, boolean>>>>()

/** 检查记忆 */
function checkMemory(sessionId: string, taskType: TaskType, modelId: string): boolean
| null {
  const session = sessionMemory.get(sessionId)
  if (!session) return null
  const tasks = session.get(taskType)
  if (!tasks) return null
  return tasks.get(modelId) ?? null
}

/** 记录记忆 */
function recordMemory(sessionId: string, taskType: TaskType, modelId: string, success:
boolean) {
  if (!sessionMemory.has(sessionId)) sessionMemory.set(sessionId, new Map())
  const session = sessionMemory.get(sessionId)!
  if (!session.has(taskType)) session.set(taskType, new Map())
  session.get(taskType)!.set(modelId, success)
}
```

路由日志系统

路由引擎内置了结构化日志系统，保留最近 500 条路由日志，支持按 sessionId、phase、level 等多维度过滤。日志通过 externalLogCallback 同步到 UI 活动栏，让用户可以实时观察路由过程：

```

export interface RouteLogEntry {
  id: string
  ts: number // 时间 (ms)
  level: RouteLogLevel
  phase: RouteLogPhase // 阶段
  sessionId: string
  message: string // 消息
  elapsed?: number // 耗时 (ms)
  data?: Record<string, unknown>
}

// 默认最大日志数 500 条
const MAX_LOGS = 500
const routeLogs: RouteLogEntry[] = []

// 日志入队函数
function rlog(level, phase, sessionId, message, elapsed?, data?) {
  const entry = { id: `rl-${++logSeq}`, ts: Date.now(), level, phase, sessionId,
    message, elapsed, data }
  routeLogs.unshift(entry)
  if (routeLogs.length > MAX_LOGS) routeLogs.length = MAX_LOGS
  if (externalLogCallback) externalLogCallback(level, phase, message, elapsed)
}

```

routeModel() 函数与 RouteResult 结构

routeModel 是路由引擎的主入口函数，其签名和返回类型如下：

```

/** 路由引擎主入口函数 */
export async function routeModel(
  sessionId: string,
  userMessage: string,
  history: ChatMsg[],
  options?: {
    taskType?: TaskType // 任务类型 detect 任务
    forceModelId?: string // 强制模型 ID
    onChunk?: (piece: string) => void // 分片回调
    onLog?: (entry: RouteLogEntry) => void // 日志回调
  }
): Promise<RouteResult>

export interface RouteResult {
  success: boolean
  content: string // 回复内容
  modelId: string // 模型 ID
  modelName: string
  taskType: TaskType // 任务类型
  elapsed: number // 耗时 (ms)
  rounds: AgentRound[] // Agent 交互轮次
  logs: RouteLogEntry[] // 日志
  error?: string
}

```

系统提示构建

buildEnhancedMessages()

函数负责构建增强的系统提示，注入个性化指令、技能上下文和工具定义：

```
/** 构建增强后的系统提示 + 用户消息 + 历史消息 */
function buildEnhancedMessages(
  userMessage: string,
  history: ChatMsg[],
  systemPrompt: string,
  skillContext: SkillContext,
  toolPrompt: string,
): ChatMsg[] {
  const messages: ChatMsg[] = []

  // 1. 构建系统提示
  let enhancedSystem = systemPrompt

  // 2. 注入个性化指令
  if (personalizedInstructions) {
    enhancedSystem += '\n\n## 个性化指令\n' + personalizedInstructions
  }

  // 3. 注入技能上下文
  if (skillContext.injectedPrompt) {
    enhancedSystem += '\n\n## 技能上下文\n' + skillContext.injectedPrompt
  }

  // 4. 注入工具提示
  if (toolPrompt) {
    enhancedSystem += '\n\n## 工具提示\n' + toolPrompt
  }

  messages.push({ role: 'system', content: enhancedSystem, ... })

  // 5. 注入历史消息 token
  messages.push(...truncateHistory(history))

  // 6. 注入用户消息
  messages.push({ role: 'user', content: userMessage, ... })

  return messages
}
```

2.3 cua.ts — 计算机控制与工具系统

工具分类：A 类（计算机控制 11 种）+ B 类（金融数据 49 种）| Agent 循环最多 5 轮

模块用途

cua.ts 实现了 AI Agent 的工具调用系统，灵感来源于 trycua/cua 项目。该模块定义了两类工具：A 类计算机控制工具（通过 IPC 桥接执行鼠标、键盘、截屏等系统操作，共 11 种）和 B 类金融数据工具（直接调用前端 API 获取行情、回测、财务等数据，共 49 种）。模型可以在回复中输出工具调用指令，系统解析后执行并将结果回传，形成 Agent 推理循环。

核心类型定义

```
export interface ToolDef {
  name: string
  category: 'cua' | 'data' // A 类 B 类
  description: string // 工具描述
  parameters: {
    name: string
    type: 'string' | 'number' | 'boolean'
    description: string
    required: boolean
  }[]
  execute: (params: Record<string, unknown>) => Promise<ToolResult>
}

export interface ToolCall {
  name: string
  params: Record<string, unknown>
  raw: string // 原始调用字符串
}

export interface ToolResult {
  success: boolean
  data?: unknown
  error?: string
  elapsed: number // 耗时 (ms)
}

export interface AgentRound {
  round: number // 轮次 1-50
  modelResponse: string // 模型回复
  toolCalls: ToolCall[] // 工具调用
  toolResults: ToolResult[] // 工具结果
  elapsed: number
}

export interface AgentResult {
  rounds: AgentRound[]
  finalResponse: string // 最终回复
  totalElapsed: number
  totalToolCalls: number
}
```

工具系统提示构建

buildToolSystemPrompt()

函数将所有可用工具的定义格式化为系统提示，注入到模型对话中。模型通过识别特定的格式指令来发起工具调用：

JSON

```
/** 格式化系统提示 */
export function buildToolSystemPrompt(tools: ToolDef[]): string {
  const cuaTools = tools.filter(t => t.category === 'cua')
  const dataTools = tools.filter(t => t.category === 'data')

  let prompt = '#####'
  prompt += '##### JSON 格式\n'
  prompt += '```json\n{"tool": "工具名", "params": {}}\n```\n\n'

  if (cuaTools.length > 0) {
    prompt += '## 工具调用\n'
    cuaTools.forEach(t => {
      prompt += ` - ${t.name}: ${t.description}\n`
      t.parameters.forEach(p => {
        prompt += ` - ${p.name} (${p.type}): ${p.description}${p.required ? ' [必填]' : ''}\n`
      })
    })
  }

  if (dataTools.length > 0) {
    prompt += '\n## 数据工具\n'
    dataTools.forEach(t => {
      prompt += ` - ${t.name}: ${t.description}\n`
    })
  }

  return prompt
}
```

工具调用解析与执行

parseToolCalls() 从模型回复中提取工具调用指令，executeDataTool() 执行 B 类金融数据工具：

```

/** 解析JSON字符串 */
export function parseToolCalls(response: string): ToolCall[] {
  const calls: ToolCall[] = []
  // 匹配 ```json ... ``` 格式
  const pattern = /```json\s*({[\s\S]*?})\s*```/g
  let match
  while ((match = pattern.exec(response)) !== null) {
    try {
      const parsed = JSON.parse(match[1])
      if (parsed.tool) {
        calls.push({ name: parsed.tool, params: parsed.params || {}, raw: match[0] })
      }
    } catch { /* 解析失败 */ }
  }
  return calls
}

/** 执行工具调用 */
async function executeDataTool(call: ToolCall): Promise<ToolResult> {
  const start = Date.now()
  const tool = DATA_TOOLS.find(t => t.name === call.name)
  if (!tool) {
    return { success: false, error: `工具: ${call.name}`, elapsed: 0 }
  }
  try {
    const data = await tool.execute(call.params)
    return { success: true, data, elapsed: Date.now() - start }
  } catch (e) {
    return { success: false, error: String(e), elapsed: Date.now() - start }
  }
}

```

Agent 循环流程

Agent 循环最多执行 5 轮，每轮包含模型推理和工具执行两个阶段：

轮次	模型推理	工具执行	终止条件
第 1 轮	分析用户请求，决定是否调用工具	执行解析到的工具调用	无工具调用 -> 返回最终回复
第 2 轮	基于工具结果继续推理	可能发起新的工具调用	无工具调用 / 达到最大轮数
第 3 轮	综合前序结果推理	执行新的工具调用	无工具调用 / 达到最大轮数
第 4 轮	深度分析或验证	可能的补充工具调用	无工具调用 / 达到最大轮数
第 5 轮	最终综合与回复生成	—（最后一轮不再调用工具）	强制终止，返回当前回复

2.4 vbtService.ts — VectorBT 服务客户端

通信层：Electron IPC / HTTP | Single-Flight Cache | 6 种策略参数网格

模块用途

vbtService.ts 是前端与 Python VectorBT 服务之间的通信客户端。它通过 Electron IPC 或 HTTP 两种方式与后端 FastAPI 服务通信，提供参数扫描、快速回测、滚动优化等功能。该模块移植了 tickflow 的 PanelCache 设计，实现了 Single-Flight Cache（并发去重 + LRU + TTL）以避免缓存雪崩和重复计算。

通信层设计

vbtFetch() 是通信层的核心函数，它根据运行环境自动选择 IPC 或 HTTP 通道：

```
/** 尝试通过 IPC 或 HTTP */
async function vbtFetch(endpoint: string, body: unknown): Promise<any> {
  // Electron 尝试 IPC
  if (typeof window !== 'undefined' && window.agentcore?.vbt) {
    return window.agentcore.vbt.request(endpoint, body)
  }
  // 尝试 HTTP
  const res = await fetch(`http://127.0.0.1:17897${endpoint}`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(body),
  })
  if (!res.ok) throw new Error(`VBT 错误: ${res.status}`)
  return res.json()
}
```

Single-Flight Cache 机制

Single-Flight Cache 是并发去重的核心。当多个请求同时发起相同的参数扫描时，只执行一次实际计算，其余请求等待结果共享。缓存还支持 LRU 淘汰和 TTL 过期：

```
// SingleFlightCache + LRU + TTL
class SingleFlightCache<K, V> {
  private cache = new Map<K, Promise<V>>() // 缓存
  private lru = new Map<K, { value: V; expire: number }>() // LRU
  private stats = { hit: 0, miss: 0, inflight: 0, eviction: 0 }
  private ttl: number // 过期时间 (ms) 默认 3 秒
  private maxSize: number // 最大缓存数

  async get(key: K, fn: () => Promise<V>): Promise<V> {
    // 1. 检查 LRU
    const cached = this.lru.get(key)
    if (cached && cached.expire > Date.now()) {
      this.stats.hit++
      return cached.value
    }
    // 2. 检查 Single-Flight
    const inflight = this.cache.get(key)
    if (inflight) {
      this.stats.hit++
      return inflight // 返回 Promise
    }
    // 3. 缓存命中
    this.stats.miss++
    this.stats.inflight++
    const promise = fn().finally(() => {
      this.cache.delete(key)
      this.stats.inflight--
    })
    this.cache.set(key, promise)
    const value = await promise
    // 4. 检查 LRU
    this.lru.set(key, { value, expire: Date.now() + this.ttl })
    if (this.lru.size > this.maxSize) {
      const oldest = this.lru.keys().next().value
      this.lru.delete(oldest)
      this.stats.eviction++
    }
    return value
  }
}
```

6 种策略参数网格预设

vbtService 预定义了 6 种策略的参数搜索空间：

策略类型	参数网格	默认范围
ma_cross (双均线交叉)	fast_period / slow_period	[5,10,15,20] / [20,30,40,60]
bollinger (布林带)	window / num_std	[10,15,20,25] / [1.5,2.0,2.5,3.0]

策略类型	参数网格	默认范围
rsi (RSI 超买超卖)	period / oversold / overbought	[7,14,21] / [20,25,30] / [70,75,80]
macd (MACD 金叉)	fast / slow / signal	[8,12,16] / [20,26,30] / [7,9,11]
turtle (海龟突破)	entry / exit / atr_period	[10,15,20] / [5,10] / [14,20]
momentum (动量策略)	lookback / threshold	[5,10,20,60] / [0.01,0.02,0.03]

健康检查

checkVBTHHealth() 检测后端服务的可用性及依赖库状态：

```
/** 检查 VBT 依赖库 */
export async function checkVBTHHealth(): Promise<VBTHHealthStatus> {
  try {
    const res = await vbtFetch('/health', {})
    return {
      healthy: res.status === 'ok',
      vectorbt: res.vectorbt ?? false,
      lightgbm: res.lightgbm ?? false,
      scipy: res.scipy ?? false,
      version: res.version,
      timestamp: Date.now(),
    }
  } catch {
    return { healthy: false, vectorbt: false, lightgbm: false, scipy: false, timestamp: Date.now() }
  }
}
```

2.5 backtest.ts — 回测策略引擎

21 个经典策略模型 | T+1 开盘价成交 | 因子引擎集成 | 手续费与印花税

模块用途

backtest.ts 是前端内置的量化回测引擎，包含 21 个经典策略模型，覆盖趋势跟踪（双均线、海龟、多头排列）、均值回归（布林带、RSI、CCI）、动量策略、多因子打分等主要策略类型。回测采用 T+1 开盘价成交模式，包含手续费（万 2.5）和印花税（千 1）的精确计算。

策略定义结构

```
export interface Strategy {
  id: string
  name: string
  desc: string
  hold: (ks: KPoint[]) => boolean[] // 持仓信号
}

// 均线交叉策略
const maCross: Strategy = {
  id: 'ma_cross',
  name: '均线交叉',
  desc: '均线交叉策略',
  hold: (ks) => {
    const closes = ks.map(k => k.close)
    const ma5 = maSeries(closes, 5)
    const ma20 = maSeries(closes, 20)
    return ks.map((_, i) => {
      if (i < 20 || ma5[i] == null || ma20[i] == null) return false
      return ma5[i]! > ma20[i]!
    })
  }
}
```

技术指标计算函数

backtest.ts 实现了完整的技术指标序列计算函数：

函数名	功能	参数
maSeries(xs, n)	简单移动平均线（SMA）	xs: 价格序列, n: 周期
emaSeries(xs, n)	指数移动平均线（EMA）	xs: 价格序列, n: 周期
smaCn(xs, n, m)	中国式 SMA（递推加权）	xs: 序列, n: 周期, m: 权重
rsiSeries(cs, n)	RSI 相对强弱指标	cs: 收盘价, n: 周期（默认 6/12/24）
macdSeries(cs)	MACD 指标（DIF + DEA）	cs: 收盘价，返回 {dif, dea}
kdjSeries(ks)	KDJ 随机指标	ks: K 线数据，返回 {k, d}
bollSeries(cs, n, mult)	布林带（中轨/上轨/下轨）	cs: 收盘价, n: 周期, mult: 标准差倍数

因子引擎集成

backtest.ts 集成了 factorEngine 的 6 种因子信号生成器，支持基于因子的策略回测：

```
// 初始化信号生成器
import {
  generateFactorSignals, // 因子信号
  generateGroundVolumeSignals, // 地面成交量信号
  generateSentimentBetaSignals, // 情绪 Beta 信号
  generateDragonTigerSentimentSignals, // 龙虎资金信号
  generateRelTurnSignals, // 相对强弱信号
  generateIntradayDivergenceSignals // 日内背离信号
} from './factorEngine'

// 增强因子策略
const factorEnhanced: Strategy = {
  id: 'factor_enhanced',
  name: '因子增强',
  desc: '结合 6 个因子信号生成增强信号',
  hold: (ks) => {
    const signals = generateFactorSignals(ks)
    return signals.map(s => s === true)
  }
}
```

2.6 dataSourceManager.ts — 数据源管理器

30+ 数据源注册 | 优先级 10-180 | 熔断机制 | 自动恢复

模块用途

dataSourceManager.ts 是统一数据源调度器，将 30+ 个数据源注册到一个管理器中，按优先级自动尝试。任何一个源失败时自动降级到下一个，全部失败时返回兜底数据。该模块还实现了熔断机制（连续失败 5 次标记不健康）和自动恢复机制（30 秒后重试）。

数据源健康跟踪

```
export interface DataSourceHealth {
  id: DataSourceId
  name: string
  healthy: boolean
  lastSuccess: number // 上次成功时间
  lastFailure: number // 上次失败时间
  failCount: number // 失败次数
  avgLatency: number // 平均延迟 (ms)
  totalCalls: number // 总调用次数
  successCalls: number // 成功调用次数
}

// 配置
const CIRCUIT_BREAKER_THRESHOLD = 5 // 熔断阈值 5 次
const RECOVERY_INTERVAL = 30000 // 30 秒恢复间隔
```

数据源优先级注册

数据源按优先级注册（数字越小优先级越高）：

优先级	数据源	类型	说明
10	eastmoney	内置 HTTP	东方财富（主源，免费行情）
20	tencent	内置 HTTP	腾讯财经（备源）
30	sina	内置 HTTP	新浪财经（备源）
40	akshare	Python 桥	AkShare 沪深京 A 股全市场
50	tushare	HTTP API	Tushare Pro（需 Token）
55	tushareMcp	MCP Server	Tushare MCP（SSE/StreamableHTTP）
60	efinance	Python 桥	efinance 东财源
70	pytdx	Python 桥	Pytdx 通达信协议
80	baostock	Python 桥	BaoStock 证券数据
90	lixinger	HTTP API	理杏仁基本面估值
100	yahoo	HTTP API	Yahoo Finance 国际兜底
110	guosen	HTTP API	国信证券（沪深京/港/美）
120	miaoxiang	HTTP API	东方财富妙想 AI
130	ptrade	HTTP API	Ptrade 量化交易平台
140	alphavantage	HTTP API	Alpha Vantage 全球行情
150	finnhub	HTTP API	Finnhub 美股行情
160	twelvedata	HTTP API	Twelve Data 全球行情
170	polygon	HTTP API	Polygon.io 聚合数据
180	kronos	本地模型	Kronos AI 预测

自动切换逻辑

```

/** 尝试切换数据源 */
export async function autoSwitch<T>(  
  task: (source: DataSourceId) => Promise<T>,  
  preferred?: DataSourceId[]  
) : Promise<AutoSwitchResult<T>> {  
  const tried: AutoSwitchResult<T>['tried'] = []  
  const sources = preferred || getHealthySources() // 获取健康数据源  
  
  for (const sourceId of sources) {  
    const health = healthMap.get(sourceId)  
    if (health && !health.healthy) {  
      // 跳过不健康的数据源  
      if (Date.now() - health.lastFailure < RECOVERY_INTERVAL) continue  
    }  
    const start = Date.now()  
    try {  
      const data = await task(sourceId)  
      const latency = Date.now() - start  
      recordSuccess(sourceId, latency)  
      tried.push({ source: sourceId, ok: true, latency })  
      return { data, source: sourceId, tried, totalElapsed: latency }  
    } catch (e) {  
      recordFailure(sourceId)  
      tried.push({ source: sourceId, ok: false, latency: Date.now() - start, error: String(e) })  
    }  
  }  
  return { data: null, source: null, tried, totalElapsed: 0 }  
}

```

2.7 marketApi.ts — 实时行情接入

双通道通信 | 智能代码识别 | secid 转换 | 东方财富 Push2

模块用途

marketApi.ts 是实时行情数据的接入层，基于东方财富 Push2 接口获取免费行情数据。该模块支持双通道通信：在 Electron 环境下走主进程 IPC（无跨域限制），在浏览器开发环境下直连（东方财富接口支持 CORS）。

智能代码识别 normalizeCode()

normalizeCode() 函数实现了智能股票代码识别，用户只需输入数字或字母即可自动判断市场：

```
// 600519 -> 600519.SH | 000858 -> 000858.SZ | 830799 -> 830799.BJ
// 00700 -> 00700.HK | AAPL -> AAPL.US
export function normalizeCode(input: string): string | null {
  const s = input.trim().toUpperCase()
  if (!s) return null
  // 验证格式
  if (/^\d{6}\.(SH|SZ|BJ)$/.test(s)) return s
  if (/^\d{1,5}\.HK$/.test(s)) return s.split('.')[0].padStart(5, '0') + '.HK'
  if (/^[A-Z]{1,5}\.US$/.test(s)) return s
  // 6 位代码A 验证格式
  if (/^\d{6}$/.test(s)) {
    if (/^(43|83|87|88|92)/.test(s)) return `${s}.BJ` // 000
    if (/^(60|68|90|5)/.test(s)) return `${s}.SH` // 00
    if (/^(00|30|20|12)/.test(s)) return `${s}.SZ` // 00
    return `${s}.SH` // 0000
  }
  // 1~5 位代码
  if (/^\d{1,5}$/.test(s)) return `${s.padStart(5, '0')}.HK`
  // 验证格式
  if (/^[A-Z]{1,5}$/.test(s)) return `${s}.US`
  return null
}
```

secid 转换 toSecid()

东方财富使用 secid 格式标识证券，toSecid() 负责标准代码到 secid 的转换：

```
// secid 格式 1. / 0. / 0. / 116. / 105.
export function toSecid(code: string, usMkt = 105): string {
  if (isSecid(code)) return code // 00 secid 00
  const [num, mkt] = code.split('.')
  switch ((mkt ?? '').toUpperCase()) {
    case 'SH': return `1.${num}` // 00
    case 'SZ': return `0.${num}` // 00
    case 'BJ': return `0.${num}` // 000
    case 'HK': return `116.${num.padStart(5, '0')}` // 00
    case 'US': return `${usMkt}.${num.toUpperCase()}` // 00
    default: return `1.${num}`
  }
}
```

fetchQuote() 和 fetchKline()

行情获取函数根据运行环境自动选择通信通道：

```
/** 获取行情 */
export async function fetchQuote(code: string): Promise<Quote | null> {
  const secid = toSecid(normalizeCode(code) || code)
  // Electron 调用 IPC
  if (window.agentcore?.market) {
    return window.agentcore.market.quotes(secid)
  }
  // 调用 API
  const url = `https://push2.eastmoney.com/api/qt/stock/get?secid=${secid}`
  const res = await fetch(url)
  const json = await res.json()
  return parseQuote(json)
}

/** 获取 K 线 */
export async function fetchKline(code: string, limit = 120): Promise<KPoint[]> {
  const secid = toSecid(normalizeCode(code) || code)
  if (window.agentcore?.market) {
    return window.agentcore.market.kline(secid, limit)
  }
  const url = `https://push2his.eastmoney.com/api/qt/stock/kline/get` +
    `?secid=${secid}&fields1=f1,f2,f3&fields2=f51,f52,f53,f54,f55,f56,f57` +
    `&klt=101&fq=1&end=20500101&limit=${limit}`
  const res = await fetch(url)
  return parseKlines(await res.json())
}
```

2.8 skillClaw.ts — 自学习技能管理

5 阶段生命周期 | 6 种技能分类 | 种子技能 SEED_VERSION=5

模块用途

skillClaw.ts 实现了 AI 自学习技能管理系统，灵感来源于 AMAP-ML/SkillClaw 项目。该模块让模型能够通过对话自动发现、提取、注入、进化和淘汰可复用的分析模式和策略。技能以结构化形式存储在 localStorage 中，在后续对话中自动检索匹配并注入系统提示。

技能生命周期

阶段	名称	说明
1	发现	模型在对话中发现可复用的分析模式或策略
2	提取	从对话中提取结构化技能（触发词 + 提示词 + 示例）
3	注入	后续对话中自动检索相关技能注入系统提示
4	进化	根据使用反馈自动优化技能提示词（version + 1）

阶段	名称	说明
5	淘汰	长期低成功率的技能 (<30%) 被标记为 deprecated

Skill 类型定义

```
export type SkillCategory = 'analysis' | 'trading' | 'research' | 'general' | 'tool' | 'data'

export interface Skill {
  id: string
  name: string
  description: string
  category: SkillCategory
  triggers: string[] // 触发词
  prompt: string // 技能提示词
  examples: SkillExample[] // 技能示例
  createdAt: string
  updatedAt: string
  usageCount: number
  successCount: number
  failCount: number
  version: number // 版本号 +1
  autoLearned: boolean // 是否自动学习 vs 手动学习
  status: 'active' | 'draft' | 'deprecated'
  sourceModel?: string // 技能来源模型
  tags?: string[]
}

// 常量
const SEED_VERSION = 5 // v5: 初始 tradingagents 技能
const MAX_SKILLS = 200 // 最多 200 个技能
const DEPRECATION_THRESHOLD = 0.3 // 成功率 30% 以下淘汰
```

技能匹配与注入

buildSkillContext()

函数根据用户输入匹配触发词，计算匹配分数，选择最相关的技能注入系统提示：

```

/** 技能匹配 + 评分 */
export function buildSkillContext(userInput: string): SkillContext {
  const allSkills = loadSkills().filter(s => s.status === 'active')
  const matches: { skill: Skill; matchedTriggers: string[]; score: number }[] = []

  for (const skill of allSkills) {
    const matchedTriggers: string[] = []
    for (const trigger of skill.triggers) {
      if (userInput.toLowerCase().includes(trigger.toLowerCase())) {
        matchedTriggers.push(trigger)
      }
    }
    if (matchedTriggers.length > 0) {
      // 评分 = 技能使用次数 * 技能成功率 + 技能匹配数
      const successRate = skill.usageCount > 0
        ? skill.successCount / skill.usageCount : 0.5
      const score = matchedTriggers.length * (0.5 + successRate * 0.5)
      matches.push({ skill, matchedTriggers, score })
    }
  }

  // 按评分排序 5 个
  matches.sort((a, b) => b.score - a.score)
  const top = matches.slice(0, 5)

  // 生成提示
  const injectedPrompt = top.map(m =>
    `### ${m.skill.name}\n${m.skill.prompt}`
  ).join('\n\n')

  return {
    skills: top.map(m => m.skill),
    injectedPrompt,
    matchDetails: top.map(m => ({
      skillId: m.skill.id,
      matchedTriggers: m.matchedTriggers,
      score: m.score
    })))
  }
}

```

2.9 kronos.ts — Kronos AI 预测

模型规格：mini(2048)/small(512)/base(512) | 零样本预测 | AAI 2026 论文

模块用途

kronos.ts 是 Kronos 金融市场 K 线基础模型的前端接入模块。Kronos 是首个开源的金融 K 线基础模型，基于 45 个全球交易所 120 亿条 K 线预训练，发表于 AAI 2026。该模块通过 Python 终端执行 Kronos 预测脚本，实现 K 线序列预测、波动率预测和合成 K

线数据生成，支持零样本（zero-shot）预测。

模型配置

```
export type KronosModelSize = 'mini' | 'small' | 'base'
export type KronosDevice = 'auto' | 'cpu' | 'cuda' | 'mps'

export interface KronosConfig {
  enabled: boolean
  pythonPath: string // Python 路径
  modelSize: KronosModelSize // 模型大小
  tokenizerName: string // Hugging Face 名称
  predictorName: string // Hugging Face 名称
  device: KronosDevice // 设备
  maxContext: number // 最大上下文
  repoPath: string // Kronos 仓库路径
  temperature: number // 温度
  topP: number // topP
  sampleCount: number // 采样次数
}

// 注释
// mini: maxContext=2048, 10000, 10000
// small: maxContext=512, 10000000
// base: maxContext=512, 10000, 10000

const DEFAULT_CONFIG: KronosConfig = {
  enabled: false,
  pythonPath: 'python3',
  modelSize: 'small',
  tokenizerName: 'NeoQuasar/Kronos-Tokenizer-base',
  predictorName: 'NeoQuasar/Kronos-small',
  device: 'auto',
  maxContext: 512,
  // ...
}
```

predict() 预测流程

predict() 函数通过调用 Python 脚本执行 Kronos 预测：

```
/** 00 Kronos K 000 */
export async function predict(
  klines: KPoint[],
  predLen: number = 10,
  config?: Partial<KronosConfig>
): Promise<KronosForecastResult> {
  const cfg = { ...getKronosConfig(), ...config }
  if (!cfg.enabled) throw new Error('Kronos 000')

  // 1. 000000JSON 000
  const inputData = klines.map(k => ({
    date: k.date, open: k.open, high: k.high,
    low: k.low, close: k.close, volume: k.volume
  }))

  // 2. 00 Python 0000
  const args = [
    cfg.pythonPath,
    path.join(cfg.repoPath, 'examples/prediction_new.py'),
    '--model', cfg.modelSize,
    '--pred_len', String(predLen),
    '--device', cfg.device,
    '--temperature', String(cfg.temperature),
    '--top_p', String(cfg.topP),
    '--sample_count', String(cfg.sampleCount),
  ]

  // 3. 00 IPC 00 Python 00
  const result = await window.agentcore.kronos.predict(inputData, args)
  return parseForecastResult(result)
}
```

2.10 knowledge.ts — 知识库系统

双系统：LLM Wiki（主）+ AnythingLLM（备） | 自动 fallback

模块用途

knowledge.ts 实现了双知识库系统，主库为 LLM Wiki（默认 192.168.3.20:19828），备库为 AnythingLLM（默认 192.168.3.20:3001）。检索时主库不通自动切换到备用库，双库共存互不干扰。请求经主进程 kb:request IPC（HTTP 仅允许 localhost/私网段，HTTPS 任意），浏览器态使用 fetch 兜底。

知识库端点配置

```
export const KB_DEFAULTS: KnowledgeEndpoint[] = [
  {
    id: 'kb-lw-primary',
    name: 'LLM Wiki',
    type: 'llmwiki',
    baseUrl: 'http://192.168.3.20:19828',
    token: 'smfFLGc3ZQRG9oR4u2wGo7z-kCFQ-E6SmAXzXFv0tzs',
    target: '9c5bf878-20d3-4c9b-8d94-78a83660a15d',
  },
  {
    id: 'kb-alm-backup',
    name: 'AnythingLLM',
    type: 'anythingllm',
    baseUrl: 'http://192.168.3.20:3001',
    token: '94NZDZ8-96R4CJN-MN5ZYFA-YZQEEP9',
    target: 'lunarcore-agent',
  },
]
```

searchKnowledge() 检索函数

searchKnowledge() 实现主备库自动 fallback 的检索逻辑：

```
/** 知识库搜索接口 */
export async function searchKnowledge(
  query: string,
  topK: number = 5
): Promise<KnowledgeResult> {
  const endpoints = loadKbEndpoints()

  for (const ep of endpoints) {
    try {
      if (ep.type === 'llmwiki') {
        // LLM Wiki 接口
        const res = await kbRequest(ep.baseUrl, '/api/search', {
          method: 'POST',
          headers: { 'Authorization': `Bearer ${ep.token}` },
          body: JSON.stringify({ query, target: ep.target, topK })
        })
        return parseLlmWikiResult(res, ep)
      } else if (ep.type === 'anythingllm') {
        // AnythingLLM 接口
        const res = await kbRequest(ep.baseUrl, '/api/v1/workspace/query', {
          method: 'POST',
          headers: { 'Authorization': `Bearer ${ep.token}` },
          body: JSON.stringify({ query, workspace: ep.target })
        })
        return parseAnythingLlmResult(res, ep)
      }
    } catch (e) {
      console.warn(`接口 ${ep.name} 失败:`, e)
      continue // 接口 fallback 下一个
    }
  }
  return { success: false, results: [], error: '知识库搜索失败' }
}
```

辅助模块

知识库系统还包含以下辅助模块：

- kbStorage.ts：知识库本地存储管理，处理端点持久化和缓存。
- kbUpload.ts：知识库文档上传，支持将 Markdown 文件写入 LLM Wiki 项目目录。
- kbAutoSetup.ts：知识库自动初始化，首次使用时自动配置默认端点。

2.11 其他重要模块

因子引擎 / 风险管理 / 订单管理 / 国际化 / 消息推送 / 多 Agent 框架 / 图表主题

factorEngine.ts — 因子处理引擎

因子处理引擎实现了 16 步因子流水线：从 K 线数据计算因子，经过预处理、特征工程、模型训练、滚动预测到绩效归因。该模块适配单标的时间序列分析（非横截面），所有计算仅用历史数据避免前视偏差。包含 6 种因子信号生成器：

- generateGroundVolumeSignals()：地面量信号，识别缩量回调后的放量突破。
- generateSentimentBetaSignals()：情绪 Beta 信号，度量个股相对市场的情绪偏离。
- generateDragonTigerSentimentSignals()：龙虎榜情绪信号，基于龙虎榜数据的资金动向。
- generateRelTurnSignals()：相对换手信号，个股换手率相对历史均值的偏离。
- generateIntradayDivergenceSignals()：日内分歧信号，日内价格与成交量的分歧度。
- generateFactorSignals()：综合因子信号，多因子加权打分。

riskManager.ts — 风险管理系统

风险管理模块提供持仓风险评估功能，包括：最大回撤计算、VaR（风险价值）估算、波动率分析、相关性矩阵、Beta 系数计算、Sharpe 比率等风险指标。支持设置止损止盈规则，监控持仓的风险暴露。

orderManager.ts — 订单管理

订单管理模块负责交易订单的创建、修改、取消和状态跟踪。支持与 Ptrade 量化交易平台的对接，实现委托下单、持仓查询、可转债列表查询等功能。

i18n.ts — 国际化系统

国际化模块支持 5 种语言：

语言代码	语言名称	状态
zh-CN	简体中文	默认语言，完整覆盖
zh-TW	繁体中文	完整覆盖
en	English	完整覆盖
ja	日本語	主要界面覆盖
ko		主要界面覆盖

notify.ts — 消息推送

消息推送模块支持 3 种通知渠道：

- 飞书（Feishu/Lark）：通过 Webhook 推送消息到飞书群或个人。

- WxPusher：微信推送服务，通过 HTTP API 推送到微信。
- LineNotify：Line 平台的消息推送服务。

tradingAgents.ts — 多 Agent 交易分析框架

多 Agent 交易分析框架实现了多个专业 Agent 协同分析的能力，包括基本面 Agent、技术面 Agent、情绪面 Agent、宏观 Agent 等。各 Agent 独立分析后汇总结果，提供多维度的投资决策参考。

chartTheme.ts — ECharts 图表主题

图表主题模块定义了 ECharts 的深色主题配置，包括颜色方案、字体、网格样式、tooltip 样式等。与 Tailwind 的 ink/gold/aurora/rose 四大色系保持一致，确保图表与界面风格统一。

第三章 Electron 主进程详解

Electron 主进程是桌面应用的核心，负责窗口管理、IPC 通信、系统操作、Python 进程托管和定时任务调度。主进程代码位于 `electron/` 目录，使用 CommonJS 模块格式（.cjs），包含 8 个核心模块。

3.1 main.cjs — 主进程入口

窗口创建（1440x900） | 安全配置 | 单实例锁 | 6 个 IPC 模块

模块用途

`main.cjs` 是 Electron 应用的主进程入口，负责创建应用窗口、管理应用生命周期、加载 IPC 模块和启动辅助服务。窗口默认尺寸为 1440x900（最小 1024x680），在 macOS 上使用 `hiddenInset` 标题栏样式，支持拖拽区域。

窗口创建函数

```
const { app, BrowserWindow, ipcMain, shell, Menu } = require('electron')
const path = require('path')
const vbtManager = require('./vbt-manager.cjs')
const { createScheduler } = require('./scheduler.cjs')
const scheduler = createScheduler()

let mainWindow = null
const isDev = !app.isPackaged

function createWindow() {
  mainWindow = new BrowserWindow({
    width: 1440, height: 900,
    minWidth: 1024, minHeight: 680,
    show: false, // ready-to-show 窗口
    title: 'LunarCore Agent',
    backgroundColor: '#f8fafc',
    titleBarStyle: process.platform === 'darwin' ? 'hiddenInset' : 'default',
    icon: path.join(__dirname, '..', 'build', 'icon.png'),
    webPreferences: {
      preload: path.join(__dirname, 'preload.cjs'),
      contextIsolation: true, // 上下文隔离
      nodeIntegration: false, // 禁用 Node 集成
      sandbox: false,
      webSecurity: !isDev, // 禁用 webSecurity API
    },
  })

  // 启动 Vite dev server 并监听
  if (isDev) {
    mainWindow.loadURL(`http://localhost:${process.env.VITE_PORT || 5200}`)
    mainWindow.webContents.openDevTools({ mode: 'detach' })
  } else {
    mainWindow.loadFile(path.join(__dirname, '..', 'dist', 'index.html'))
  }

  mainWindow.once('ready-to-show', () => mainWindow.show())
  mainWindow.webContents.setWindowOpenHandler(({ url }) => {
    shell.openExternal(url) // 打开外部链接
    return { action: 'deny' }
  })
  mainWindow.on('closed', () => { mainWindow = null })
}
```

单实例锁与生命周期管理

```
// 初始化锁
const gotLock = app.requestSingleInstanceLock()
if (!gotLock) {
  app.quit()
} else {
  app.on('second-instance', () => {
    if (mainWindow) {
      if (mainWindow.isMinimized()) mainWindow.restore()
      mainWindow.focus()
    }
  })

  app.whenReady().then(async () => {
    createWindow()
    // 6 个 IPC 桥
    require('./ipc-ollama.cjs') // Ollama 桥
    require('./ipc-market.cjs') // 桥
    require('./ipc-kb.cjs') // 桥
    require('./ipc-vbt.cjs') // VBT 桥
    require('./cua-bridge.cjs') // 桥
    // VBT 初始化
    vbtManager.ensureAndStart()
    scheduler.start()
  })

  app.on('before-quit', async () => {
    scheduler.stop()
    await vbtManager.stop() // 桥 VBT 桥
  })
}
```

安全配置说明

配置项	值	说明
contextIsolation	true	隔离渲染进程与 Node.js 上下文，防止原型污染
nodeIntegration	false	禁用渲染进程直接访问 Node.js API
sandbox	false	关闭沙箱（preload 脚本需要有限 Node 访问）
webSecurity	生产 true / 开发 false	生产环境启用同源策略，开发环境关闭以直连 API
titleBarStyle	hiddenInset (macOS)	隐藏标题栏，保留交通灯按钮

3.2 preload.cjs — IPC 桥

contextBridge | 6 大 API 命名空间 | 流式通信

模块用途

preload.cjs 是 Electron 的预加载脚本，通过 `contextBridge.exposeInMainWorld()` 将安全的 API 暴露给渲染进程的 `window.agentcore` 对象。该脚本在 `contextIsolation` 模式下运行，确保渲染进程无法直接访问 Node.js API，只能通过预定义的接口与主进程通信。

agentcore 对象结构

agentcore 对象包含 6 大 API 命名空间：

```

const { contextBridge, ipcRenderer } = require('electron')

// 1. chunk 初始化
const chunkCallbacks = new Map()

// 2. Ollama 初始化
ipcRenderer.on('ollama:chunk', (_event, reqId, piece) => {
  for (const cb of chunkCallbacks.values()) cb(reqId, piece)
})
ipcRenderer.on('ollama:done', (_event, reqId) => {
  for (const cb of chunkCallbacks.values()) cb(reqId, null) // null = 完成
})

contextBridge.exposeInMainWorld('agentcore', {
  // 1. Ollama 接口
  ollama: {
    tags: (base) => ipcRenderer.invoke('ollama:tags', base),
    chat: (reqId, base, model, messages, options) =>
      ipcRenderer.invoke('ollama:chat', { reqId, base, model, messages, options }),
    onChunk: (callback) => { // 初始化
      const id = ++cbSeq
      chunkCallbacks.set(id, callback)
      return () => chunkCallbacks.delete(id) // 清理
    },
  },

  // 2. 市场数据
  market: {
    post: (url, body) => ipcRenderer.invoke('market:post', { url, body }),
    get: (url, headers) => ipcRenderer.invoke('market:get', { url, headers }),
    quotes: (secids) => ipcRenderer.invoke('market:quotes', { secids }),
    kline: (secid, limit) => ipcRenderer.invoke('market:kline', { secid, limit }),
    fundflow: (secids) => ipcRenderer.invoke('market:fundflow', { secids }),
  },

  // 3. 知识库
  kb: {
    request: (base, kbPath, options) => ipcRenderer.invoke('kb:request', {...}),
    writeFile: (dirPath, fileName, content) => ipcRenderer.invoke('kb:writeFile', {...}),
    mkdir: (dirPath) => ipcRenderer.invoke('kb:mkdir', { dirPath }),
    stat: (dirPath) => ipcRenderer.invoke('kb:stat', { dirPath }),
    listDir: (dirPath) => ipcRenderer.invoke('kb:listDir', { dirPath }),
  },

  // 4. VBT 接口

```

```
vbt: {
  request: (endpoint, body) => ipcRenderer.invoke('vbt:request', { endpoint, body }),
  health: () => ipcRenderer.invoke('vbt:health'),
},

// 5. 自定义Cua
cua: {
  click: (x, y, button) => ipcRenderer.invoke('cua:click', { x, y, button }),
  hotkey: (keys) => ipcRenderer.invoke('cua:hotkey', { keys }),
  screenshot: () => ipcRenderer.invoke('cua:screenshot'),
  // ... 11 个
},

// 6. Kronos
kronos: {
  predict: (data, args) => ipcRenderer.invoke('kronos:predict', { data, args }),
},
})
```

流式通信机制

Ollama 流式聊天使用事件驱动模式：主进程通过 `ipcRenderer.send()` 发送 `ollama:chunk` 事件，preload 脚本监听该事件并分发给已注册的回调函数。每个回调通过 `reqId` 过滤，确保多条流式消息不会串流。`piece` 为 `null` 表示流结束，`piece` 为 `undefined` 且 `error` 存在表示错误。

3.3 cua-bridge.cjs — 计算机控制

11 种系统操作 | macOS AppleScript + Windows PowerShell | 截屏 desktopCapturer

模块用途

`cua-bridge.cjs` 在 Electron 主进程中执行计算机操作，包括鼠标控制（点击、双击、右键、拖拽、滚动）、键盘控制（按键、热键、输入文本）、截屏、文件操作和终端执行。macOS 使用 AppleScript (`osascript`) 实现，Windows 使用 PowerShell 调用 Win32 API，截屏使用 Electron 的 `desktopCapturer` API。

`click()` 鼠标点击实现

```
function mouseClicked(x, y, button = 'left') {
  if (isMac) {
    // macOS: AppleScript
    const btn = button === 'right' ? 'right click'
      : button === 'double' ? 'double click' : 'click'
    const script = `
tell application "System Events"
  set position of the mouse to ${x}, ${y}
  delay 0.05
  ${btn}
end tell
`

    execSync(`osascript -e '${script}'`, { timeout: 5000 })
    return { x, y, button }
  } else if (isWin) {
    // Windows: PowerShell + Win32 API
    const script = `
Add-Type -AssemblyName System.Windows.Forms
[System.Windows.Forms.Cursor]::Position = New-Object System.Drawing.Point(${x}, ${y})
Start-Sleep -Milliseconds 50
`

    execSync(`powershell -Command "${script}"`, { timeout: 5000 })
    return { x, y, button }
  }
  throw new Error('不支持的操作')
}
```

hotkey() 热键实现

```
function hotkey(keys) {
  // keys: ['cmd', 'shift', 'p']
  if (isMac) {
    // macOS: AppleScript
    const mods = keys.filter(k => ['cmd', 'shift', 'ctrl', 'alt'].includes(k))
    const normal = keys.filter(k => !mods.includes(k))
    let script = `tell application "System Events" to key code`
    // 通过 AppleScript 实现
    execSync(`osascript -e '${script}'`, { timeout: 5000 })
  } else if (isWin) {
    // Windows: PowerShell SendKeys
    const sendKeys = keys.map(k => mapToSendKey(k)).join('')
    execSync(`powershell -Command
"[System.Windows.Forms.SendKeys]::SendWait('${sendKeys}')"`)
  }
}
```

11 种系统操作列表

操作	IPC 通道	说明
click	cua:click	鼠标点击（左键/右键/双击）

操作	IPC 通道	说明
move	cua:move	鼠标移动到指定坐标
drag	cua:drag	鼠标拖拽（从 A 点到 B 点）
scroll	cua:scroll	鼠标滚轮滚动
hotkey	cua:hotkey	组合键按下（如 Cmd+Shift+P）
type	cua:type	输入文本字符串
screenshot	cua:screenshot	截取屏幕图片（desktopCapturer）
openApp	cua:openApp	打开应用程序
runCommand	cua:runCommand	执行终端命令
readFile	cua:readFile	读取文件内容
writeFile	cua:writeFile	写入文件内容

3.4 vbt-manager.cjs — VBT 进程管理

5 种环境检测 | 自动安装 | 健康监控（60 秒） | 优雅退出

模块用途

vbt-manager.cjs 负责管理 Python VectorBT 服务的完整生命周期，包括环境检测、自动安装、启动服务、健康监控和优雅退出。该模块确保 VBT 服务在各种环境下（打包/开发/无 Python）都能正常运行或优雅降级。

5 种环境检测模式

模式	条件	Python 来源	说明
packaged	app.asar 内含 PyInstaller 产物	打包的可执行文件	生产模式，无需外部 Python
dev-bundled	python/.venv 存在且可用	venv 中的 python	开发模式，已安装 venv
dev-venv	系统有 Python 但无 venv	系统 Python 创建 venv	首次开发，自动创建 venv

模式	条件	Python 来源	说明
dev-system	系统有 Python，直接使用	系统 python3/python	无 venv，直接用系统 Python
none	系统无 Python	—	VBT 不可用，优雅降级

ensureAndStart() 启动流程

```

/** 初始化 VBT 环境 */
async function ensureAndStart(onLog) {
  const log = (level, msg) => onLog && onLog(level, msg)

  // Step 1: 检查环境
  const env = checkEnvironment(onLog)
  if (env.mode === 'none') {
    log('error', '系统无 Python 无法启动 VBT')
    return { success: false, error: 'no-python' }
  }

  // Step 2: 安装依赖
  if (env.mode === 'dev-venv') {
    log('info', '正在安装依赖...')
    const installResult = await autoInstall(env, onLog)
    if (!installResult.success) {
      log('error', '安装失败: ' + installResult.error)
      return installResult
    }
  }

  // Step 3: 启动服务
  log('info', '启动 VBT 服务...')
  const startResult = await startVBTSvc(env, onLog)
  if (startResult.success) {
    // Step 4: 启动健康检查 60 秒一次
    startHealthCheck()
  }
  return startResult
}

```

自动安装流程（3 步）

- Step 1：创建 venv — 使用系统 Python 创建虚拟环境（python -m venv .venv）。
- Step 2：安装依赖 — 执行 pip install -r requirements.txt 安装 vectorbt、fastapi 等。
- Step 3：验证 — 检查 venv 中的 Python 是否可用，验证关键库是否安装成功。

健康监控

健康监测每 60 秒执行一次，通过 HTTP 请求 /health 端点检测服务状态。如果连续 3 次健康检查失败，会尝试重启服务。

优雅退出

```
/** 退出 VBT 逻辑 */
async function stop() {
  // 清理定时器
  if (healthCheckTimer) {
    clearInterval(healthCheckTimer)
    healthCheckTimer = null
  }
  // 尝试优雅退出 SIGTERM -> 3s -> SIGKILL
  if (vbtProcess) {
    return new Promise((resolve) => {
      vbtProcess.kill('SIGTERM')
      const forceKill = setTimeout(() => {
        try { vbtProcess.kill('SIGKILL') } catch {}
        resolve()
      }, 3000) // 3 秒后强制退出
      vbtProcess.on('exit', () => {
        clearTimeout(forceKill)
        vbtProcess = null
        resolve()
      })
    })
  }
}
```

3.5 scheduler.cjs — 交易调度器

node-schedule | A 股交易时段调度 | 自动采集任务

模块用途

scheduler.cjs 使用 node-schedule 实现基于 cron 表达式的定时任务调度。该模块根据 A 股交易时段（9:30-11:30, 13:00-15:00）自动执行数据采集任务，包括行情快照采集、资金流向更新、异动监控等。

调度规则

```
const schedule = require('node-schedule')

function createScheduler() {
  let jobs = []

  function start() {
    // A 09:15 000000
    jobs.push(schedule.scheduleJob('15 9 * * 1-5', () => {
      collectAuctionData()
    }))

    // 00000000 5 000000
    jobs.push(schedule.scheduleJob('* /5 9-11 * * 1-5', () => {
      collectMarketData()
    }))

    // 00000000 5 000000
    jobs.push(schedule.scheduleJob('* /5 13-14 * * 1-5', () => {
      collectMarketData()
    }))

    // 000015:05 000000
    jobs.push(schedule.scheduleJob('5 15 * * 1-5', () => {
      collectEndOfDayData()
    }))
  }

  function stop() {
    jobs.forEach(job => job.cancel())
    jobs = []
  }

  return { start, stop }
}
```

第四章 Python 后端服务

Python 后端服务位于 `python/` 目录，是量化回测和 AI 预测的核心计算引擎。该服务基于 FastAPI 框架构建，通过 HTTP API 为前端提供向量化回测、参数优化、因子训练和 K 线预测等计算密集型服务。后端与 Electron 主进程解耦，通过 `vbt-manager.cjs` 自动管理生命周期。

4.1 vbt_server.py — FastAPI 量化服务

端口 127.0.0.1:17897 | 4 个核心端点 | 6 种策略 | PyInstaller 打包适配

模块用途

vbt_server.py 是基于 FastAPI 的量化回测 HTTP 服务，为 TypeScript 前端提供向量化回测与参数优化能力。服务监听 127.0.0.1:17897 端口，支持 CORS 跨域。当 vectorbt 未安装时，服务自动降级到 fallback 模式，返回错误信息而不崩溃。该服务还适配 PyInstaller 打包模式，处理打包后路径和缓存目录的问题。

服务初始化与 PyInstaller 适配

```
#!/usr/bin/env python3
import os, sys, tempfile
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

# PyInstaller 适配
IS_FROZEN = getattr(sys, 'frozen', False)

if IS_FROZEN:
# 适配numba/llvmlite 路径
_cache_base = os.path.join(os.path.expanduser('~'), '.investment-ai', 'vbt-cache')
os.makedirs(_cache_base, exist_ok=True)
os.environ.setdefault('NUMBA_CACHE_DIR', _cache_base)
os.environ.setdefault('NUMBA_PYCC_DIR', _cache_base)
os.environ.setdefault('LLVM_TEMP_DIR', tempfile.gettempdir())

import numpy as np
import pandas as pd

try:
import vectorbt as vbt
VBT_AVAILABLE = True
except ImportError:
VBT_AVAILABLE = False
print("WARNING: vectorbt not installed. Service will run in fallback mode.")

app = FastAPI(title="VectorBT Bridge", version="1.0.0")
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"],
allow_headers=["*"])
```

请求/响应模型

```
class KLineData(BaseModel):
    date: str
    open: float
    high: float
    low: float
    close: float
    volume: float
    amount: Optional[float] = 0.0

class ParamScanRequest(BaseModel):
    """参数扫描"""
    klines: List[KLineData]
    strategy_type: str # ma_cross | bollinger | rsi | macd | turtle | momentum
    param_grid: Dict[str, List[float]] # 参数网格
    init_cash: float = 100000.0
    fees: float = 0.00025 # 费率2.5
    stamp_duty: float = 0.001 # 印花税1
    metric: str = "sharpe_ratio" # 指标

class QuickBacktestRequest(BaseModel):
    """快速回测"""
    klines: List[KLineData]
    strategy_type: str
    params: Dict[str, float]
    init_cash: float = 100000.0
    fees: float = 0.00025
    stamp_duty: float = 0.001
```

核心 API 端点

端点	方法	功能	说明
/param_scan	POST	参数扫描	遍历参数网格，返回最优参数组合和 Top 结果
/walk_forward	POST	滚动优化	Walk-Forward 滚动窗口训练+测试
/quick_backtest	POST	快速回测	单次参数回测，返回完整指标和权益曲线
/health	GET	健康检查	检测 vectorbt/LightGBM/SciPy 可用性

参数扫描端点实现

```

@app.post("/param_scan")
async def param_scan(req: ParamScanRequest):
    if not VBT_AVAILABLE:
        raise HTTPException(503, "vectorbt not installed")

    # 创建 DataFrame
    df = pd.DataFrame([k.dict() for k in req.klines])
    df.set_index('date', inplace=True)

    # 生成组合
    from itertools import product
    keys = list(req.param_grid.keys())
    values = list(req.param_grid.values())
    combinations = list(product(*values))

    results = []
    for combo in combinations:
        params = dict(zip(keys, combo))
        # 运行回测
        metrics = run_backtest(df, req.strategy_type, params,
                               req.init_cash, req.fees, req.stamp_duty)
        results.append({
            'params': params,
            'metric_value': metrics[req.metric],
            'total_return': metrics['total_return'],
            'sharpe': metrics['sharpe_ratio'],
            'max_drawdown': metrics['max_drawdown'],
            'win_rate': metrics['win_rate'],
        })

    # 获取 Top 10
    results.sort(key=lambda x: x['metric_value'], reverse=True)
    best = results[0]

    return {
        'status': 'ok',
        'strategy_type': req.strategy_type,
        'total_combinations': len(combinations),
        'scanned': len(results),
        'best_params': best['params'],
        'best_metric_value': best['metric_value'],
        'top_results': results[:10],
        'metric': req.metric,
    }

```

降级机制

当 vectorbt 导入失败时，VBT_AVAILABLE 设为 False。此时所有 API 端点会返回 503 错误，前端 vbtService 通过 checkVBHealth() 检测到不可用状态后，自动回退到前端内置的 backtest.ts 回测引擎。这种降级机制确保应用在缺少 Python 环境时仍能提供基本功能。

4.2 lgbm_factors.py — LightGBM 因子模型

因子计算与训练 | Walk-Forward 滚动窗口 | 滚动预测

模块用途

`lgbm_factors.py` 使用 LightGBM 梯度提升树模型进行因子信号训练和预测。该模块实现了 Walk-Forward 滚动窗口训练方案，在每个时间窗口内使用历史数据训练模型，然后在下一个窗口进行预测，避免前视偏差。

因子计算与训练接口

```

import lightgbm as lgb
import numpy as np
import pandas as pd

def compute_factors(klines: pd.DataFrame) -> pd.DataFrame:
    """计算因子"""
    df = klines.copy()
    # 收益率
    df['returns_1d'] = df['close'].pct_change(1)
    df['returns_5d'] = df['close'].pct_change(5)
    df['returns_20d'] = df['close'].pct_change(20)
    # 波动率
    df['volatility_20d'] = df['returns_1d'].rolling(20).std()
    # 成交量比
    df['volume_ratio'] = df['volume'] / df['volume'].rolling(20).mean()
    # RSI
    df['rsi_14'] = compute_rsi(df['close'], 14)
    # MACD
    df['macd'], df['macd_signal'] = compute_macd(df['close'])
    return df

def train_lgbm_model(X_train, y_train, params=None):
    """LightGBM 训练"""
    default_params = {
        'objective': 'binary',
        'num_leaves': 31,
        'learning_rate': 0.05,
        'feature_fraction': 0.8,
        'bagging_fraction': 0.8,
        'bagging_freq': 5,
        'verbose': -1,
    }
    params = params or default_params
    train_data = lgb.Dataset(X_train, label=y_train)
    model = lgb.train(params, train_data, num_boost_round=100)
    return model

def walk_forward_train(df, features, target, window_size=252, step=21):
    """Walk-Forward 训练"""
    results = []
    for start in range(0, len(df) - window_size - step, step):
        train = df.iloc[start : start + window_size]
        test = df.iloc[start + window_size : start + window_size + step]
        model = train_lgbm_model(train[features], train[target])
        predictions = model.predict(test[features])

        results.append({
            'train_start': train.index[0],
            'train_end': train.index[-1],
            'test_start': test.index[0],
            'test_end': test.index[-1],
            'predictions': predictions,
            'actual': test[target].values,
        })
    return results

```

4.3 Kronos/ — AI 预测模型

模型架构：Tokenizer + Predictor | 零样本预测 | AAAI 2026

模型概述

Kronos 是首个开源的金融 K 线基础模型，发表于 AAAI 2026。模型基于 45 个全球交易所 120 亿条 K 线数据预训练，具备零样本（zero-shot）K 线预测能力。Kronos 的架构由两个核心组件构成：Tokenizer（将连续的 OHLCV 数据离散化为 token 序列）和 Predictor（基于 Transformer 的自回归预测器）。

模型架构

组件	功能	说明
Tokenizer	OHLCV -> Token	将连续 K 线数据编码为离散 token 序列
Predictor	Token -> 预测 Token	基于 Transformer 的自回归序列预测
Decoder	Token -> OHLCV	将预测 token 解码回 OHLCV 价格序列

预测调用流程

```
# Kronos 初始化
import torch
from model.kronos import KronosModel

# 1. 加载模型
model = KronosModel.from_pretrained(
    tokenizer_name='NeoQuasar/Kronos-Tokenizer-base',
    predictor_name='NeoQuasar/Kronos-small',
    device='auto' # 指定 GPU/CPU
)

# 2. 加载 K 线数据
klines = pd.DataFrame({
    'open': [...], 'high': [...],
    'low': [...], 'close': [...],
    'volume': [...]
})

# 3. 进行预测
predictions = model.predict(
    klines,
    prediction_length=10, # 预测 10 根 K 线
    temperature=0.7, # 采样温度
    top_p=0.9, # top-p 采样
    sample_count=5 # 5 次采样
)

# 4. 打印结果
for pred in predictions:
    print(f"{pred['date']}: O={pred['open']:.2f} H={pred['high']:.2f} "
          f"L={pred['low']:.2f} C={pred['close']:.2f}")
```

模型规格对比

规格	上下文长度	参数量	速度	精度	适用场景
mini	2048	最小	最快	较低	实时预测、快速验证
small	512	中等	中等	中等	日常分析、平衡选择
base	512	最大	最慢	最高	深度研究、精度优先

Kronos 目录结构

Kronos 模块包含以下子目录：

- model/：核心模型代码（kronos.py + module.py），定义 Tokenizer 和 Predictor。
- examples/：预测示例脚本（prediction_example.py、prediction_batch_example.py 等）。
- finetune/：微调工具（train_tokenizer.py、train_predictor.py）。

- finetune_csv/：CSV 数据微调工具，支持自定义 K 线数据微调。
- webui/：Web UI 界面，提供可视化的预测交互。
- tests/：回归测试，验证模型输出的一致性。

第五章 前端页面架构

前端页面位于 `src/pages/` 目录，共 29 个页面组件。应用采用状态驱动路由（非 `react-router`），通过 `Zustand` `store` 的 `page` 字段控制当前显示的页面。本章将深入解析核心页面的架构设计、状态管理和数据流。

5.1 App.tsx — 应用根组件

状态驱动路由 | PAGES 映射表 (25 个页面 key) | macOS 拖拽区域

模块用途

`App.tsx` 是应用的根组件，负责页面路由、布局组合和窗口交互。它通过 `Zustand store` 的 `page` 字段确定当前页面，从 `PAGES` 映射表中查找对应的组件进行渲染。布局由 `Background`（动态背景）、`Sidebar`（侧边栏）和 `main`（主内容区）三部分组成。

PAGES 映射结构

```
import React from 'react'
import { useStore } from './lib/store'
import Background from './components/Background'
import Sidebar from './components/Sidebar'
import Overview from './pages/Overview'
import Chat from './pages/Chat'
// ... 其他页面

const PAGES: Record<string, React.ComponentType> = {
  overview: Overview,
  chat: Chat,
  market: Market,
  screener: Screener,
  fundflow: FundFlow,
  sector: Sector,
  collector: DataCollector,
  prediction: Prediction,
  alerts: AlertCenter,
  quant: Quant,
  portfolio: Portfolio,
  commodities: Commodities,
  fundmarket: FundMarket,
  globalmarket: GlobalMarket,
  newsmonitor: NewsMonitor,
  industriechain: IndustryChain,
  journal: Journal,
  notify: Notify,
  plugin: Plugin,
  review: DailyReview,
  research: Research,
  news: News,
  knowledge: Knowledge,
  settings: Settings,
  privacy: Privacy,
}

export default function App() {
  const { page } = useStore()
  const PageComponent = PAGES[page] ?? Overview

  return (
    <div className="h-full w-full flex relative">
      <Background />
      <Sidebar />
      <main className="flex-1 h-full overflow-hidden">
```

```
{/* macOS 窗口 */}
<div className="absolute top-0 right-0 h-7 z-40"
style={{ left: '15rem', WebkitAppRegion: 'drag' }} />
<div key={page} className="h-full animate-fade-in">
  <PageComponent />
</div>
</main>
</div>
)
```

布局结构

App 组件的布局由三个主要部分组成：

- Background：动态背景组件，提供 5 层流光极光带和鼠标跟随光斑效果。
- Sidebar：侧边栏导航组件，包含 22 个导航项和状态指示灯。
- main：主内容区，渲染当前页面组件，使用 animate-fade-in 动画过渡。
- 拖拽区域：macOS 顶部 28px 高的拖拽区域（WebkitAppRegion: 'drag'），跳过侧栏宽度。

5.2 Chat.tsx — AI 对话核心

多模型路由日志 | 上下文注入 | Agent 工具调用循环 | Token 管理

模块用途

Chat.tsx 是 AI 对话的核心页面，集成了模型路由引擎、Agent 工具调用循环和上下文注入系统。用户发送消息后，页面调用 `routeModel()` 进行智能路由，流式展示模型回复，并在模型调用工具时实时显示工具调用过程和结果。

消息发送流程

```

async function handleSend() {
  const userMsg = input.trim()
  if (!userMsg || sending) return

  // 1. 初始化
  const userMsgObj = { id: uid(), role: 'user', content: userMsg, time: now() }
  setMessages(prev => [...prev, userMsgObj])
  setInput('')
  setSending(true)

  // 2. 构建上下文/提示词
  const context = await buildChatContext(userMsg)
  const enrichedMsg = context ? `${userMsg}\n\n[context]\n${context}` : userMsg

  // 3. 路由
  const sessionId = currentSessionId
  const result = await routeModel(sessionId, enrichedMsg, messages, {
    onChunk: (piece) => {
      // 更新回复
      setStreamingReply(prev => prev + piece)
    },
    onLog: (entry) => {
      // 记录日志
      setRouteLogs(prev => [entry, ...prev])
    }
  })

  // 4. 更新消息
  setMessages(prev => [...prev, {
    id: uid(), role: 'assistant', content: result.content,
    time: now(), model: result.modelName
  }])
  setStreamingReply('')
  setSending(false)
}

```

Token 管理

Chat.tsx 实现了 Token 估算和历史截断功能，控制上下文窗口大小：

```
// 估计 token 数量 1 个中文字符 ~= 2 token, 1 个英文字符 ~= 1.3 token
function estimateTokens(text: string): number {
  const chinese = (text.match(/[\u4e00-\u9fa5]/g) || []).length
  const english = (text.match(/[a-zA-Z]/g) || []).length
  return Math.ceil(chinese * 2 + english * 1.3)
}

// 截断历史记录
function truncateHistory(messages: ChatMsg[], maxTokens: number = 4000): ChatMsg[] {
  let totalTokens = 0
  const result: ChatMsg[] = []
  // 从最新消息开始倒序遍历
  for (let i = messages.length - 1; i >= 0; i--) {
    const tokens = estimateTokens(messages[i].content)
    if (totalTokens + tokens > maxTokens) break
    totalTokens += tokens
    result.unshift(messages[i])
  }
  return result
}
```

上下文注入

buildChatContext() 函数根据用户消息内容，自动注入相关的上下文数据，包括当前持仓信息、最近回测结果、实时行情快照等，让模型能够基于最新数据进行分析。

5.3 Quant.tsx — 量化回测（最复杂页面）

6 种回测模式 | VBT 参数扫描 UI | 因子引擎集成 | 风险管理系统

模块用途

Quant.tsx 是应用中最复杂的页面，集成了前端回测引擎和 Python VBT 服务。该页面支持 6 种回测模式，每种模式有独立的 UI 和逻辑分支。页面还集成了因子引擎和风险管理系统，提供完整的量化分析 workflow。

6 种回测模式

模式	名称	说明
single	单策略回测	选择 21 个内置策略之一，使用默认参数回测
ai	AI 智能回测	使用 AI 模型分析 K 线数据，自动推荐策略和参数
multi	多策略对比	同时运行多个策略，对比收益率/回撤/夏普等指标

模式	名称	说明
vbt	VBT 参数扫描	调用 Python vectorbt 进行参数网格搜索，找到最优参数
portfolio	组合回测	多标的组合回测，支持仓位分配和再平衡
framework	算法框架	AI/多策略/板块轮动等高级框架模式

quantMode 分支逻辑

```
function renderQuantContent() {
  switch (quantMode) {
    case 'single':
      return <SingleBacktestPanel
        strategies={STRATEGIES}
        klines={klines}
        onRun={runSingleBacktest}
      />
    case 'ai':
      return <AIBacktestPanel
        klines={klines}
        onAnalyze={runAIBacktest}
      />
    case 'multi':
      return <MultiStrategyPanel
        strategies={STRATEGIES}
        klines={klines}
        onCompare={runMultiBacktest}
      />
    case 'vbt':
      return <VBTParamScanPanel
        klines={klines}
        strategyTypes={VBT_STRATEGIES}
        onScan={runParamScan}
        vbtHealth={vbtHealth}
      />
    case 'portfolio':
      return <PortfolioBacktestPanel
        holdings={holdings}
        onRun={runPortfolioBacktest}
      />
    case 'framework':
      return <AlgorithmFrameworkPanel
        frameworkType={frameworkType}
        onRun={runFramework}
      />
  }
}
```

5.4 Settings.tsx — 系统设置

11 个 Tab 页 | 模型配置面板 | 个性化指令编辑器 | 量化引擎状态

11 个 Tab 页

Tab	名称	功能
1	模型管理	配置 AI 模型（API/本地/聚合）、端点、API Key、能力
2	路由规则	配置任务类型到模型的映射规则和偏好权重
3	级联配置	降级策略、最大重试次数、级联记忆开关
4	数据源	30+ 数据源的启用/禁用、Token 配置、优先级
5	知识库	LLM Wiki/AnythingLLM 端点管理
6	量化引擎	VBT 服务状态、Kronos 模型配置
7	个性化指令	系统提示编辑器、自定义指令管理
8	通知设置	飞书/WxPusher/LineNotify 配置
9	技能管理	自学习技能查看、编辑、删除
10	插件管理	Skill/MCP 插件安装、权限配置
11	关于	版本信息、更新日志、隐私政策

5.5 DataCollector.tsx — 数据采集器

双 Tab（行情/指标） | 股票池三种模式 | 知识库自动同步

模块用途

DataCollector.tsx 提供批量数据采集功能，支持行情数据和指标数据两种采集模式。股票池支持三种模式：手动输入、自选股列表、自定义筛选条件。采集到的数据可以自动同步到知识库，用于后续的 AI 分析和回测。

采集流程

```
async function startCollection() {
// 1. 初始化
const pool = await getStockPool(poolMode)
// poolMode: 'manual' | 'watchlist' | 'screener'

// 2. 并发获取数据
const CONCURRENCY = 5
const results = await concurrentMap(pool, CONCURRENCY, async (code) => {
try {
const klines = await fetchKline(code, 250)
const quote = await fetchQuote(code)
return { code, success: true, data: { klines, quote } }
} catch (e) {
return { code, success: false, error: String(e) }
}
})

// 3. 过滤成功结果
const success = results.filter(r => r.success)
const failed = results.filter(r => !r.success)

// 4. 同步到知识库
if (autoSyncKB) {
for (const r of success) {
const md = formatKlineToMarkdown(r.data)
await syncToKnowledgeBase(r.code, md)
}
}
}
```

5.6 Prediction.tsx — 价格预测

线性回归/MA 外推/蒙特卡洛/Kronos | 预测叠加图

4 种预测方法

方法	原理	适用场景
线性回归	最小二乘法拟合趋势线	短期趋势预测
MA 外推	移动平均线外推	平滑趋势预测
蒙特卡洛	GBM 几何布朗运动模拟（1000 路径）	概率分布预测
Kronos	AI 基础模型零样本预测	深度学习预测

蒙特卡洛 GBM 模拟

```
// 蒙特卡罗GBM模拟
function monteCarloGBM(
  prices: number[],
  days: number,
  paths: number = 1000
): number[][] {
  const returns = prices.slice(1).map((p, i) =>
    Math.log(p / prices[i])
  )
  const mu = returns.reduce((a, b) => a + b, 0) / returns.length
  const sigma = Math.sqrt(
    returns.reduce((a, b) => a + (b - mu) ** 2, 0) / returns.length
  )
  const lastPrice = prices[prices.length - 1]
  const dt = 1 / 252 // 日

  const allPaths: number[][] = []
  for (let p = 0; p < paths; p++) {
    const path: number[] = [lastPrice]
    for (let d = 0; d < days; d++) {
      // GBM:  $S(t+1) = S(t) * \exp((\mu - \sigma^2/2) * dt + \sigma * \sqrt{dt} * Z)$ 
      const z = gaussianRandom() // 高斯随机数
      const nextPrice = path[d] * Math.exp(
        (mu - sigma * sigma / 2) * dt + sigma * Math.sqrt(dt) * z
      )
      path.push(nextPrice)
    }
    allPaths.push(path)
  }
  return allPaths
}
```

第六章 UI 组件与样式系统

UI 组件位于 `src/components/` 目录，包含 4 个核心组件。样式系统基于 Tailwind CSS 3.4，配合自定义的 `ink/gold/aurora/rose` 四大色系、5 种自定义动画和 Glassmorphism 组件类，打造出深色科技风格的视觉体验。

6.1 Sidebar.tsx — 侧边栏导航

22 个导航项 | 流光玻璃背景 | 状态指示灯

模块用途

Sidebar.tsx 是应用的侧边栏导航组件，包含 22 个导航项，覆盖全部功能页面。侧边栏使用流光玻璃背景效果（Glassmorphism），底部显示 VBT

服务和知识库的状态指示灯。

22 个导航项

序号	导航项	页面 Key	功能
1	概览	overview	市场总览与自选股
2	AI 对话	chat	智能投资助手
3	行情	market	实时行情与 K 线
4	选股器	screener	条件筛选选股
5	资金流向	fundflow	主力资金监控
6	板块	sector	板块涨跌分析
7	数据采集	collector	批量数据采集
8	预测	prediction	价格预测
9	预警中心	alerts	价格异动预警
10	量化回测	quant	策略回测与优化
11	持仓	portfolio	持仓管理
12	商品	commodities	大宗商品行情
13	基金	fundmarket	基金市场
14	全球市场	globalmarket	全球指数
15	新闻监控	newsmonitor	新闻舆情
16	产业链	industriechain	产业链分析
17	日志	journal	交易日志
18	通知	notify	消息推送配置
19	插件	plugin	插件管理
20	复盘	review	每日复盘
21	研究	research	研报与知识库
22	设置	settings	系统设置

6.2 Chart.tsx — ECharts 封装

dark 主题 | canvas 渲染 | ResizeObserver 自适应

模块用途

Chart.tsx 是对 ECharts 的 React 封装组件，提供统一的图表渲染接口。组件使用 dark 主题和 canvas 渲染模式，通过 ResizeObserver 实现自适应容器大小。支持 K 线图、折线图、柱状图、面积图等多种图表类型。

```
import ReactECharts from 'echarts-for-react'
import * as echarts from 'echarts'

// 注册主题
echarts.registerTheme('lunarcore-dark', {
  backgroundColor: 'transparent',
  textStyle: { color: '#a0aec0' },
  // ... 其他配置
})

export default function Chart({ option, style, onEvents }) {
  const ref = useRef(null)

  // ResizeObserver 监听
  useEffect(() => {
    const observer = new ResizeObserver(() => {
      ref.current?.getEchartsInstance().resize()
    })
    if (ref.current?.ele) observer.observe(ref.current.ele)
    return () => observer.disconnect()
  }, [])

  return (
    <ReactECharts
      ref={ref}
      option={option}
      theme="lunarcore-dark"
      opts={{ renderer: 'canvas' }}
      style={{ height: '100%', ...style }}
      onEvents={onEvents}
    />
  )
}
```

6.3 Background.tsx — 动态背景

5 层流光极光带 | 鼠标跟随光斑 | 玻璃折射网格

模块用途

Background.tsx 实现了应用的动态背景效果，包含 5 层流光极光带（不同颜色和动画速度）、鼠标跟随光斑（径向渐变光晕）和玻璃折射网格（微妙的网格纹理）。这些效果通过 CSS 动画和 React 状态管理实现，不影响应用性能。

5 层极光带配置

层级	颜色	动画	说明
1	aurora-500 (#1ab8a8)	float 6s	底部青绿色极光带
2	gold-500 (#d4a843)	float 8s	中下部金色极光带
3	rose-500 (#e04458)	float 10s	中部玫红色极光带
4	aurora-600 (#0d9085)	float 12s	中上部深青色极光带
5	ink-700 (#1c2338)	float 14s	顶部深蓝灰色极光带

6.4 Tailwind 配置系统

ink/gold/aurora/rose 四大色系 | 自定义动画 | Glassmorphism

四大色系定义

```
// tailwind.config.js
colors: {
  ink: { // 靛青系
    950: '#070a12', 900: '#0b0f1a', 850: '#0f1422',
    800: '#141a2b', 700: '#1c2338', 600: '#252d45',
    500: '#323b57',
  },
  gold: { // 金色系
    400: '#e8c468', 500: '#d4a843', 600: '#b88a2a',
  },
  aurora: { // 极光系
    400: '#3dd9c8', 500: '#1ab8a8', 600: '#0d9085',
  },
  rose: { // 玫瑰系
    400: '#f06a7a', 500: '#e04458',
  },
}
```

自定义动画

```
animation: {
  'fade-up': 'fadeUp 0.6s cubic-bezier(0.16, 1, 0.3, 1) forwards',
  'fade-in': 'fadeIn 0.4s ease forwards',
  'glow': 'glow 3s ease-in-out infinite alternate',
  'shimmer': 'shimmer 2s linear infinite',
  'float': 'float 6s ease-in-out infinite',
},
keyframes: {
  fadeUp: { '0%': { opacity: 0, transform: 'translateY(20px)' }, '100%': { opacity: 1, transform: 'translateY(0)' } },
  fadeIn: { '0%': { opacity: 0 }, '100%': { opacity: 1 } },
  glow: { '0%': { boxShadow: '0 0 20px rgba(61,217,200,0.1)' }, '100%': { boxShadow: '0 0 40px rgba(61,217,200,0.25)' } },
  shimmer: { '0%': { backgroundPosition: '-200% 0' }, '100%': { backgroundPosition: '200% 0' } },
  float: { '0%,100%': { transform: 'translateY(0)' }, '50%': { transform: 'translateY(-20px)' } },
}
```

6.5 字体系统

Noto Serif SC (标题) | Noto Sans SC (正文) | JetBrains Mono (数据/代码)

字体配置

```
// tailwind.config.js
fontFamily: {
  display: ['"Noto Serif SC"', 'Georgia', 'serif'], // 标题
  sans: ['"Noto Sans SC"', 'system-ui', 'sans-serif'], // 正文
  mono: ['"JetBrains Mono"', 'monospace'], // 数据/代码
}
```

字体用途说明：

- Noto Serif SC：用于页面标题和重要文字，衬线字体提供优雅的视觉效果。
- Noto Sans SC：用于正文和 UI 文字，无衬线字体保证可读性。
- JetBrains Mono：用于数据展示、代码块和数字，等宽字体确保对齐。

第七章 数据流程与通信

本章梳理

LunarCore

Agent

中四条核心数据流的完整链路，从用户操作到最终结果展示，涵盖前端、Electron 主进程和 Python 后端的交互全流程。

7.1 实时行情数据流

用户操作 -> marketApi -> IPC -> 主进程 -> HTTP -> 东方财富 -> 返回 -> 渲染

完整链路

步骤	组件	操作	数据格式
1	用户界面	输入股票代码或点击自选股	字符串（如 600519）
2	marketApi.ts	normalizeCode() 智能识别市场	标准代码（如 600519.SH）
3	marketApi.ts	toSecid() 转换为东财格式	secid（如 1.600519）
4	marketApi.ts	检查 window.agentcore 可用性	—
5	preload.cjs	ipcRenderer.invoke('marke t:quotes')	IPC 消息
6	ipc-market.cjs	构建东方财富 API URL	HTTP URL
7	主进程	net.fetch() 发送 HTTP 请求	HTTP GET
8	东方财富服务器	返回 JSON 行情数据	JSON
9	ipc-market.cjs	解析 JSON，提取价格/量额	Quote 对象
10	preload.cjs	IPC 返回结果到渲染进程	Quote 对象
11	React 组件	更新状态，渲染行情卡片/K 线图	UI 更新

在浏览器开发环境下（无 Electron），步骤 5-10 替换为直接 fetch 东方财富 API。由于东方财富 Push2 接口支持 CORS，浏览器直连不会遇到跨域问题。

7.2 AI 对话数据流

用户输入 -> modelRouter 路由 -> 模型 API -> 流式回复 -> 工具调用 -> 结果回传

完整链路

步骤	组件	操作
1	Chat.tsx	用户输入消息，点击发送

步骤	组件	操作
2	Chat.tsx	buildChatContext() 构建上下文（行情/回测/持仓）
3	modelRouter.ts	routeModel() 启动 9 阶段路由管线
4	modelRouter.ts	detect: 检测任务类型（chat/analysis/code）
5	modelRouter.ts	filter: 按能力筛选可用模型
6	modelRouter.ts	memory: 查询会话记忆缓存
7	modelRouter.ts	preference + rule: 计算偏好分数，匹配路由规则
8	modelRouter.ts	buildEnhancedMessages() 注入技能+工具提示
9	ollama.ts / API	调用选定模型（流式响应）
10	modelRouter.ts	onChunk 回调流式更新 UI
11	cua.ts	parseToolCalls() 检测工具调用
12	cua.ts	executeDataTool() 执行金融数据工具
13	modelRouter.ts	工具结果回传模型，进入下一轮
14	modelRouter.ts	达到最大轮数或无工具调用，返回最终回复
15	Chat.tsx	添加回复到消息列表，更新路由日志面板

7.3 量化回测数据流

K 线数据 -> vbtService -> IPC -> 主进程 -> HTTP -> Python FastAPI -> vectorbt

完整链路

步骤	组件	操作
1	Quant.tsx	用户选择策略和参数网格
2	Quant.tsx	获取 K 线数据（fetchKline）
3	vbtService.ts	构建 VBTParamScanRequest
4	vbtService.ts	SingleFlightCache 检查缓存
5	vbtService.ts	vbtFetch() 发送请求
6	preload.cjs	ipcRenderer.invoke('vbt:request')

步骤	组件	操作
7	ipc-vbt.cjs	HTTP POST 到 127.0.0.1:17897/param_scan
8	vbt_server.py	解析请求，构建 DataFrame
9	vbt_server.py	遍历参数组合，vectorbt 回测
10	vbt_server.py	计算指标（Sharpe/回撤/胜率）
11	vbt_server.py	排序取 Top 10，返回 JSON
12	ipc-vbt.cjs	IPC 返回结果
13	vbtService.ts	写入 SingleFlightCache（TTL 3 分钟）
14	Quant.tsx	渲染参数扫描结果表格和图表

7.4 数据采集流程

股票池 -> 并发请求 -> 各数据源 -> 结果合并 -> 知识库同步

完整链路

步骤	组件	操作
1	DataCollector.tsx	选择股票池模式（手动/自选/筛选）
2	DataCollector.tsx	获取股票代码列表
3	autoCollector.ts	并发请求（CONCURRENCY=5）
4	dataSourceManager.ts	autoSwitch() 按优先级尝试数据源
5	marketApi.ts	fetchKline + fetchQuote
6	各数据源	东方财富/腾讯/新浪/AkShare 等
7	dataSourceManager.ts	记录健康状态，失败自动降级
8	DataCollector.tsx	合并结果，统计成功/失败
9	kbUpload.ts	格式化为 Markdown
10	knowledge.ts	同步到 LLM Wiki/AnythingLLM

第八章 配置与部署

本章详述项目的配置文件和部署流程，包括 `package.json`、`vite.config.ts`、`tailwind.config.js` 和 Python 环境配置。

8.1 package.json 关键配置

依赖版本 | Electron 构建 | 脚本命令

核心依赖

依赖	版本	类型	用途
react	19.2.7	dependencies	UI 框架
react-dom	19.2.7	dependencies	React DOM 渲染
zustand	5.0.14	dependencies	全局状态管理
echarts	6.1.0	dependencies	图表库
echarts-for-react	3.0.6	dependencies	ECharts React 封装
lucide-react	1.27.0	dependencies	图标库
motion	12.43.0	dependencies	动画库
iconv-lite	0.6.3	dependencies	字符编码转换
electron	33.4.11	devDependencies	桌面应用框架
electron-builder	25.1.8	devDependencies	打包工具
vite	8.1.1	devDependencies	构建工具
typescript	6.0.2	devDependencies	类型系统
tailwindcss	3.4.19	devDependencies	CSS 框架
vitest	4.1.10	devDependencies	测试框架
oxlint	1.71.0	devDependencies	代码检查

Electron 构建配置

```
"build": {
  "appId": "lunarcore.agent",
  "productName": "LunarCore Agent",
  "directories": { "output": "release" },
  "files": [
    "dist/**/*",
    "electron/**/*",
    "python/**/*",
    "build/**/*"
  ],
  "mac": {
    "category": "public.app-category.finance",
    "target": ["dmg", "zip"],
    "icon": "build/icon.icns"
  },
  "win": {
    "target": ["nsis"],
    "icon": "build/icon.ico"
  }
}
```

脚本命令

命令	说明
npm run dev	启动 Vite 开发服务器（端口 5200）
npm run build	TypeScript 编译 + Vite 打包
npm run electron:dev	启动 Vite + Electron 开发模式
npm run electron:build	打包 Electron 应用
npm run build:python	构建 Python 打包 bundle
npm run build:mac	完整 macOS 构建（Python + Electron）
npm run build:win	完整 Windows 构建
npm run build:all	同时构建 macOS + Windows
npm run test	运行 Vitest 测试
npm run deploy	部署脚本

8.2 vite.config.ts

开发服务器端口 5200 | Electron 插件配置

关键配置

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5200, // 端口
    strictPort: true, // 严格端口
  },
  build: {
    outDir: 'dist',
    sourcemap: false, // 是否生成 sourcemap
    rollupOptions: {
      external: ['electron'], // Electron 依赖
    },
  },
  resolve: {
    alias: {
      '@': '/src',
    },
  },
})
```

8.3 tailwind.config.js

darkMode: 'class' | 自定义颜色系 | 自定义动画

完整配置概览

```

/** @type {import('tailwindcss').Config} */
export default {
  darkMode: 'class', // 使用 class 模式
  content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],
  theme: {
    extend: {
      colors: {
        ink: { 950: '#070a12', 900: '#0b0f1a', /* ... */ },
        gold: { 400: '#e8c468', 500: '#d4a843', 600: '#b88a2a' },
        aurora: { 400: '#3dd9c8', 500: '#1ab8a8', 600: '#0d9085' },
        rose: { 400: '#f06a7a', 500: '#e04458' },
      },
      fontFamily: {
        display: ['"Noto Serif SC"', 'Georgia', 'serif'],
        sans: ['"Noto Sans SC"', 'system-ui', 'sans-serif'],
        mono: ['"JetBrains Mono"', 'monospace'],
      },
      animation: {
        'fade-up': 'fadeUp 0.6s cubic-bezier(0.16,1,0.3,1) forwards',
        'fade-in': 'fadeIn 0.4s ease forwards',
        'glow': 'glow 3s ease-in-out infinite alternate',
        'shimmer': 'shimmer 2s linear infinite',
        'float': 'float 6s ease-in-out infinite',
      },
    },
  },
  // keyframes 定义
},
},
plugins: [],
}

```

8.4 Python 环境配置

requirements.txt 依赖 | setup.sh 安装流程 | PyInstaller 打包

requirements.txt 依赖清单

```

# python/requirements.txt
vectorbt>=0.26.0 # 向量操作
fastapi>=0.104.0 # Web 框架
uvicorn>=0.24.0 # ASGI 服务器
numpy>=1.24.0 # 科学计算
pandas>=2.0.0 # 数据分析
lightgbm>=4.1.0 # 梯度提升树
scipy>=1.11.0 # 科学计算
scikit-learn>=1.3.0 # 机器学习
torch>=2.1.0 # Kronos 深度学习
einops>=0.7.0 # 张量操作
huggingface_hub>=0.20.0 # 模型下载
pydantic>=2.5.0 # 数据验证

```

setup.sh 安装流程

```
#!/bin/bash
# python/setup.sh - 安装 Python 环境

set -e
cd "$(dirname "$0")"

# 1. 安装 Python
PYTHON=${PYTHON:-python3}
if ! command -v $PYTHON &> /dev/null; then
    echo "Error: Python 3 not found"
    exit 1
fi

# 2. 创建虚拟环境
if [ ! -d ".venv" ]; then
    echo "Creating virtual environment..."
    $PYTHON -m venv .venv
fi

# 3. 安装依赖
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt

# 4. 运行测试
python -c "import vectorbt; import fastapi; print('Setup complete!')"

echo "Python environment ready."
```

PyInstaller 打包配置

Python 后端通过 PyInstaller 打包为独立可执行文件，嵌入到 Electron 应用的 resources 目录中。打包配置 (vbt_server.spec) 处理了以下关键问题：

- numba/llvmlite 缓存目录：打包后 __file__ 路径失效，通过 sys.executable 定位，缓存指向用户可写目录。
- 数据文件包含：确保 model/ 目录下的 Kronos 模型文件被打包。
- 隐藏导入：显式声明 vectorbt、numba 等库的隐式依赖。
- onedir 模式：使用目录模式而非单文件，便于增量更新和调试。

文档总结

本文档详细解析了 LunarCore Agent v2.3.0 的完整源码架构与实现细节，覆盖了前端 React 应用、Electron 主进程、Python 后端服务三大模块。项目采用三层解耦架构，通过 IPC 桥和 HTTP

API 实现跨进程通信，集成了 30+ 数据源、9 阶段 AI 模型路由、5 阶段自学习技能系统、21 个回测策略、6 种因子信号、Kronos AI 预测等丰富的金融分析能力。

LunarCore Agent 是一个功能完备的智能投资助手，适合个人投资者和量化研究者使用。项目的模块化和自动降级机制确保了在各种环境下都能稳定运行，同时保持了良好的可扩展性。